

Outcome Ledger MCP Server — Product Requirements Document

1. Executive Summary

Product: Outcome Ledger MCP Server (local agent) **Version:** 1.0.0

Status: Ready for Development **Author:** Aamir **Date:** June 2026

Purpose: A local MCP server that customers install on their infrastructure. It extracts per-user AI tool usage data (Claude Code, Cursor, GitHub Copilot) and PR outcome data (GitHub), then syncs it to the Outcome Ledger cloud API. This replaces manual CSV uploads with automated, per-user data extraction.

Key Value Proposition: - One command setup - Per-user attribution (know WHO spent WHAT) - Automated daily sync - No manual CSV uploads - Works with existing AI tools

2. Goals & Non-Goals

2.1 Goals

ID	Goal	Success Metric
G1	Extract per-user usage from Claude Code	User-level cost + token data per sync
G2	Extract per-user usage from Cursor	User-level cost + token data per sync
G3		User-level usage + acceptance data

ID	Goal	Success Metric
	Extract per-user usage from GitHub Copilot	
G4	Extract PR outcomes from GitHub	Merged PR data with revert detection
G5	Map users to teams/org	Email → GitHub username → team mapping
G6	Sync data to Outcome Ledger cloud	< 5 min sync for 90 days of data
G7	Cache data locally	Works offline, syncs when online
G8	Tenant isolation	Each MCP instance serves one tenant

2.2 Non-Goals (v1)

- Real-time streaming (batch sync only)
- Web UI (CLI only)
- Multi-tenant MCP (one tenant per installation)
- Token-level tracing (aggregate only)
- Prompt storage (no prompts stored)

3. User Stories

3.1 Setup

ID	Story	Acceptance Criteria
US-1	As a CTO, I want to install the MCP server with one command	<code>pip install outcome-ledger-mcp</code> works

ID	Story	Acceptance Criteria
US-2	As a CTO, I want to configure my API keys once	<code>outcome-ledger-mcp configure</code> saves all keys securely
US-3	As a CTO, I want to verify my setup works	<code>outcome-ledger-mcp status</code> shows green for all configured sources

3.2 Data Extraction

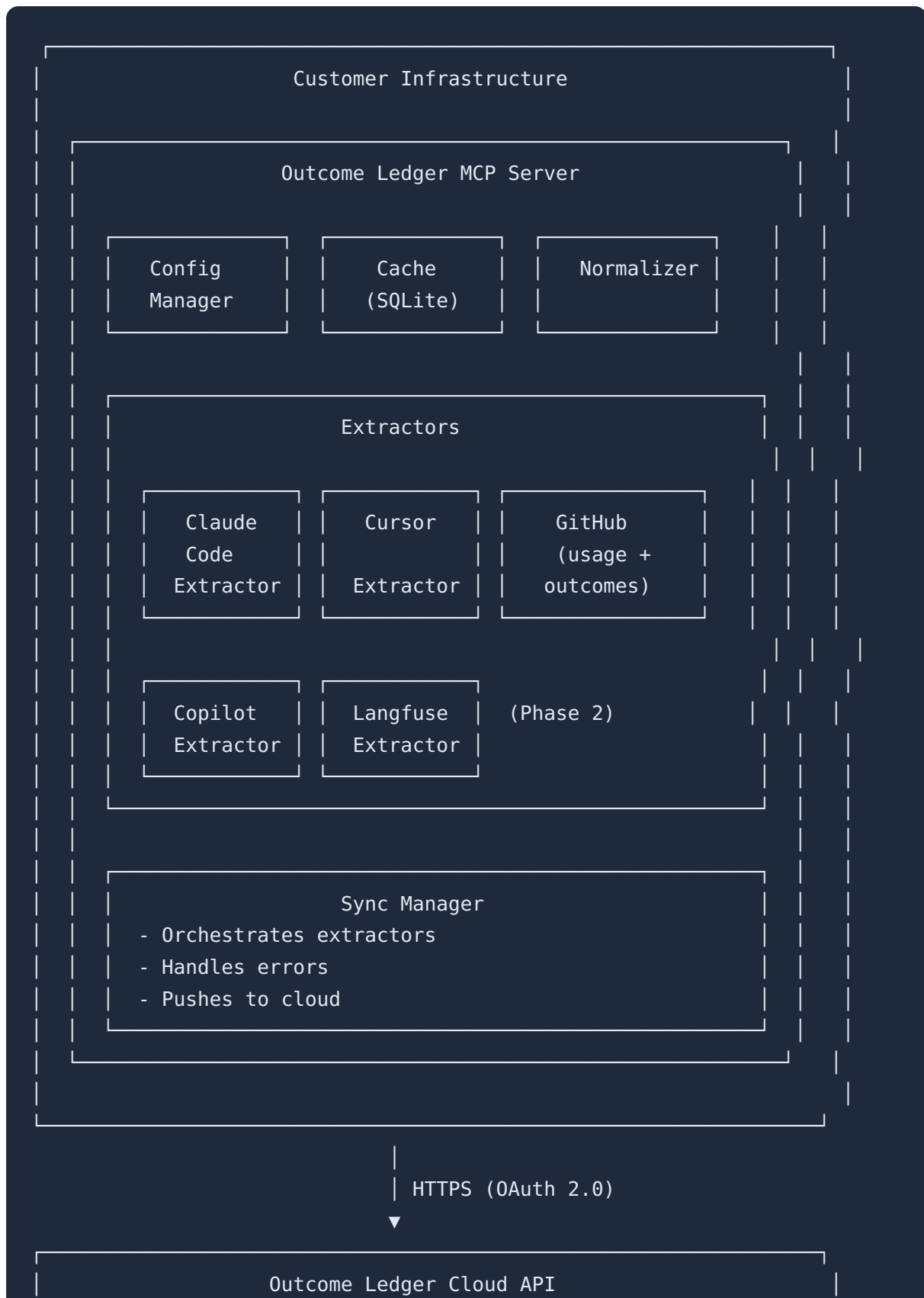
ID	Story	Acceptance Criteria
US-4	As a CTO, I want to sync all AI tool data	<code>outcome-ledger-mcp sync --since 30d</code> extracts and uploads all data
US-5	As a CTO, I want per-user attribution	Each usage event has <code>user_email</code> , <code>cost</code> , <code>tokens</code>
US-6	As a CTO, I want automated daily sync	Cron job runs <code>sync --since 1d</code> every morning

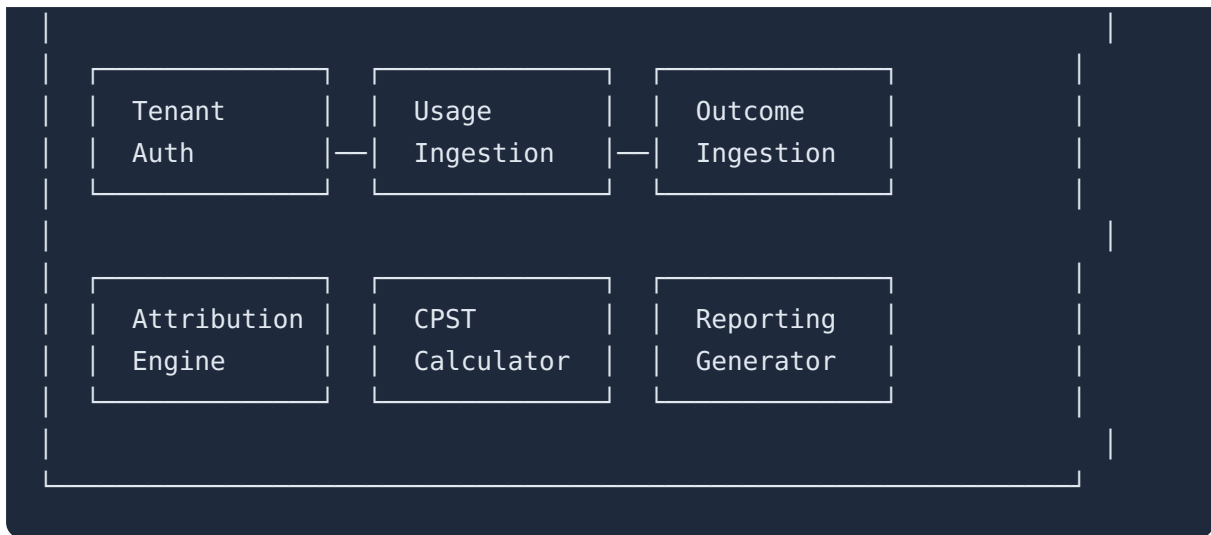
3.3 Error Handling

ID	Story	Acceptance Criteria
US-7	As a CTO, I want to know if sync fails	Error messages are clear and logged
US-8	As a CTO, I want cached data to persist	Data survives restarts, syncs when connection restored

4. Architecture

4.1 System Architecture





4.2 Tenant Isolation

Each MCP server instance serves exactly one tenant. The tenant is identified by the `outcome-ledger-key` (JWT token).

MCP Server Startup

- |
- ├─ Load config (tenant-specific)
- ├─ Validate tenant key with cloud API
- ├─ Start server with tenant context
- └─ All data tagged with `tenant_id` from key

4.3 Data Flow

Command: `outcome-ledger-mcp sync --since 30d`

- |
- ▼
- SyncManager.sync_all()
 - |
 - ├─ For each extractor:
 - | ── Test connection
 - | ── Extract usage events
 - | ── Extract outcome events
 - | └─ Normalize events
 - |
 - ├─ Cache all events locally (SQLite)
 - |
 - ├─ Push to cloud API:
 - | ── POST `/v1/ingest/usage`
 - |

```
| └─ POST /v1/ingest/outcomes
|
└─ Return sync summary
```

5. API Specification

5.1 Cloud API Endpoints (Outcome Ledger)

POST /v1/ingest/usage

Authentication: Bearer token (outcome-ledger-key)

Request:

```
{
  "tenant_id": "uuid",
  "events": [
    {
      "event_id": "uuid",
      "source": "claude_code",
      "user_email": "dev@company.com",
      "cost_usd": 45.50,
      "input_tokens": 50000,
      "output_tokens": 10000,
      "model": "claude-sonnet-4",
      "period_start": "2026-06-01T00:00:00Z",
      "period_end": "2026-06-30T23:59:59Z",
      "team_id": null,
      "repo": null,
      "metadata": {}
    }
  ]
}
```

Response:

```
{
  "status": "success",
  "ingested": 1,
```

```
"errors": []  
}
```

POST /v1/ingest/outcomes

Request:

```
{  
  "tenant_id": "uuid",  
  "events": [  
    {  
      "outcome_id": "uuid",  
      "source": "github",  
      "outcome_type": "pr_merged_stable",  
      "accepted": true,  
      "occurred_at": "2026-06-15T14:00:00Z",  
      "repo": "company/service",  
      "pr_number": 123,  
      "author": "devusername",  
      "team_id": "platform-team",  
      "metadata": {  
        "title": "Add checkout flow",  
        "additions": 120,  
        "deletions": 30,  
        "reverted": false  
      }  
    }  
  ]  
}
```

GET /v1/health

Response:

```
{  
  "status": "healthy",  
  "version": "1.0.0"  
}
```

POST /v1/tenants/validate

Request:

```
{
  "key": "ol_live_xxxxx"
}
```

Response:

```
{
  "valid": true,
  "tenant_id": "uuid",
  "tenant_name": "Acme Corp",
  "plan": "pilot",
  "expires_at": "2026-09-01T00:00:00Z"
}
```

GET /v1/tenants/{tenant_id}/sync-status

Response:

```
{
  "last_sync": "2026-06-04T10:00:00Z",
  "usage_events_total": 1523,
  "outcome_events_total": 456,
  "sources": {
    "claude_code": {"last_sync": "2026-06-04T10:00:00Z", "status": "ok"},
    "cursor": {"last_sync": "2026-06-04T10:00:00Z", "status": "ok"},
    "github": {"last_sync": "2026-06-04T10:00:00Z", "status": "ok"}
  }
}
```

6. Data Models

6.1 Usage Event

Field	Type	Required	Description
event_id	UUID	Yes	Unique identifier
source	string	Yes	claude_code , cursor , copilot , github
user_email	string	Yes	Email of the user
cost_usd	float	Yes	Cost in USD
input_tokens	int	Yes	Input token count
output_tokens	int	Yes	Output token count
model	string	Yes	Model name (e.g., claude-sonnet-4)
period_start	datetime	Yes	Start of the period
period_end	datetime	Yes	End of the period
team_id	string	No	Team identifier
repo	string	No	Repository name
metadata	object	No	Source-specific data

6.2 Outcome Event

Field	Type	Required	Description
outcome_id	UUID	Yes	Unique identifier
source	string	Yes	github (v1)
outcome_type	string	Yes	

Field	Type	Required	Description
			<code>pr_merged_stable</code> , <code>default_branch_commit</code>
accepted	boolean	Yes	Whether the outcome was accepted
occurred_at	datetime	Yes	When the outcome occurred
repo	string	Yes	Repository name (format: <code>owner/repo</code>)
pr_number	int	No	PR number (if applicable)
author	string	Yes	GitHub username
team_id	string	No	Team identifier
metadata	object	No	Source-specific data

6.3 Configuration

Field	Type	Required	Description
outcome_ledger_url	string	Yes	Cloud API URL
outcome_ledger_key	string	Yes	API key (JWT)
claude_key	string	No	Claude Code API key
cursor_key	string	No	Cursor API key
github_token	string	No	GitHub personal access token
github_repos	list	No	List of repos to track
sync_interval_hours	int	No	Auto-sync interval (default: 24)

7. CLI Specification

7.1 Commands

```
outcome-ledger-mcp <command> [options]
```

Command	Description	Options
configure	Set up credentials	--outcome-ledger-url , --outcome-ledger-key , --claude-key , --cursor-key , --github-token , --github-repos
sync	Sync data to cloud	--since <duration> , --dry-run , --source <name>
status	Show sync status	--verbose
test	Test all connections	--source <name>
reset	Clear local cache	--confirm
server	Start MCP server	--port <port>
version	Show version	

7.2 Examples

```
# Configure
outcome-ledger-mcp configure \
  --outcome-ledger-url https://api.outcome-ledger.com \
  --outcome-ledger-key ol_live_abc123 \
  --claude-key sk-ant-xxx \
  --cursor-key curs_xxx \
  --github-token ghp_xxx \
  --github-repos "acme/frontend,acme/backend"
```

```
# Sync last 30 days
outcome-ledger-mcp sync --since 30d

# Sync only Claude data
outcome-ledger-mcp sync --since 7d --source claude_code

# Dry run (don't upload)
outcome-ledger-mcp sync --since 30d --dry-run

# Check status
outcome-ledger-mcp status --verbose

# Test connections
outcome-ledger-mcp test

# Start MCP server
outcome-ledger-mcp server

# Clear cache
outcome-ledger-mcp reset --confirm
```

7.3 Exit Codes

Code	Meaning
0	Success
1	General error
2	Configuration error
3	Connection error
4	Sync error
5	Invalid arguments

8. Error Handling

8.1 Error Types

Error	Cause	Resolution
<code>ConfigNotFound</code>	No configuration file	Run <code>configure</code> first
<code>InvalidKey</code>	API key is invalid	Check key, run <code>test</code>
<code>ConnectionFailed</code>	Can't reach API	Check internet, retry
<code>RateLimit</code>	Too many requests	Wait, retry with backoff
<code>DataError</code>	Invalid data format	Check source data
<code>TenantExpired</code>	Subscription expired	Contact support

8.2 Retry Logic

```
For each API call:
├─ Try 1: Immediate
├─ Try 2: Wait 5 seconds
├─ Try 3: Wait 30 seconds
├─ Try 4: Wait 2 minutes
└─ Fail: Log error, continue to next source
```

8.3 Logging

```
# Log levels
INFO: Normal operations (sync started, events extracted)
WARNING: Non-critical issues (rate limit, partial data)
ERROR: Critical failures (connection failed, auth error)
DEBUG: Detailed debugging info (--verbose flag)

# Log format
[2026-06-04 10:00:00] INFO: Starting sync for tenant xxx
[2026-06-04 10:00:01] INFO: Extracting Claude Code usage (30d)
[2026-06-04 10:00:05] INFO: Extracted 150 usage events
```

```
[2026-06-04 10:00:06] INFO: Pushing to cloud API  
[2026-06-04 10:00:08] INFO: Sync complete. 150 events uploaded.
```

9. Security

9.1 Credential Storage

- Configuration stored in `~/.outcome-ledger/config.json`
- File permissions set to `0600` (owner read/write only)
- API keys never logged or printed
- Keys transmitted over HTTPS only

9.2 Tenant Isolation

- All requests include tenant ID from API key
- Local cache is per-installation (no cross-tenant data)
- Cloud API validates tenant on every request

9.3 Data Privacy

- No prompts or completions stored
 - Only aggregate usage data (tokens, costs, timestamps)
 - GitHub data limited to PR metadata (no code content)
-

10. Testing

10.1 Unit Tests

Test	Description
<code>test_config_save_load</code>	Config persists correctly
<code>test_config_permissions</code>	File permissions are 0600
<code>test_usage_event_model</code>	UsageEvent serializes correctly
<code>test_outcome_event_model</code>	OutcomeEvent serializes correctly
<code>test_normalizer</code>	Data normalization works
<code>test_cache_store_retrieve</code>	SQLite cache works
<code>test_sync_manager</code>	Sync orchestration works
<code>test_retry_logic</code>	Retry on failure works

10.2 Integration Tests

Test	Description
<code>test_claude_extractor_real</code>	Extract from real Claude API (mocked)
<code>test_cursor_extractor_real</code>	Extract from real Cursor API (mocked)
<code>test_github_extractor_real</code>	Extract from real GitHub API (mocked)
<code>test_cloud_api_push</code>	Push data to cloud API (mocked)
<code>test_full_sync_flow</code>	End-to-end sync test

10.3 Manual Testing Checklist

- [] Install with `pip install`

- [] Configure with all sources
 - [] Run `status` — shows all green
 - [] Run `test` — all connections pass
 - [] Run `sync --since 30d` — data extracted and uploaded
 - [] Run `sync` again — no duplicates
 - [] Run `sync --dry-run` — no data uploaded
 - [] Disconnect internet — data cached locally
 - [] Reconnect — cached data uploaded
 - [] Run `reset` — cache cleared
 - [] Wrong API key — clear error message
 - [] Missing config — prompts to run configure
-

11. Project Structure

```
outcome-ledger-mcp/
├─ README.md
├─ LICENSE
├─ pyproject.toml
├─ setup.py
├─ Makefile
├─ .gitignore
├─ .github/
│   └─ workflows/
│       ├── test.yml
│       └─ publish.yml
├─ src/
│   └─ outcome_ledger_mcp/
│       ├── __init__.py
│       ├── main.py           # CLI entry point
│       ├── config.py        # Configuration management
│       ├── cache.py         # SQLite cache
│       ├── sync.py          # Sync orchestrator
│       ├── normalizer.py    # Data normalization
│       ├── server.py        # MCP server (for Claude Desktop integration)
│       ├── api_client.py    # Outcome Ledger cloud API client
│       ├── extractors/
│       │   └─ __init__.py
```



```

├── base.py                # Base extractor class
├── claude_code.py        # Claude Code extractor
├── cursor.py             # Cursor extractor
├── copilot.py            # GitHub Copilot extractor
├── github.py             # GitHub outcomes extractor
├── models/
│   ├── __init__.py
│   ├── usage_event.py    # Usage event model
│   └── outcome_event.py  # Outcome event model
├── auth/
│   ├── __init__.py
│   └── tenant.py         # Tenant authentication
├── utils/
│   ├── __init__.py
│   ├── logger.py        # Logging setup
│   ├── retry.py         # Retry logic
│   └── validators.py     # Input validation
├── tests/
│   ├── __init__.py
│   ├── conftest.py
│   ├── test_config.py
│   ├── test_cache.py
│   ├── test_sync.py
│   ├── test_normalizer.py
│   ├── test_models.py
│   ├── test_extractors.py
│   └── test_api_client.py
├── docs/
│   ├── setup.md
│   ├── troubleshooting.md
│   └── development.md
├── scripts/
│   ├── build.sh
│   ├── publish.sh
│   └── test.sh

```

12. Development Phases

Phase 1: Core (Week 1-2)

Task	Description	Status
Project setup	Repo, pyproject.toml, Makefile	
Config management	Save/load/validate config	
Cache layer	SQLite cache for local storage	
Base extractor	Abstract base class for all extractors	
Claude Code extractor	Extract per-user usage from Claude	
GitHub extractor	Extract PR outcomes from GitHub	
Normalizer	Normalize events from all sources	
Sync manager	Orchestrate extraction and upload	
API client	Client for Outcome Ledger cloud API	
CLI (configure/status/sync)	Basic CLI commands	
Unit tests	Test core components	

Phase 2: Additional Sources (Week 3)

Task	Description	Status
Cursor extractor	Extract per-user usage from Cursor	
Copilot extractor	Extract per-user usage from Copilot	
User mapping	Email → GitHub username mapping	
Integration tests	Test with real APIs (mocked)	

Task	Description	Status
Documentation	README, setup guide	

Phase 3: Polish (Week 4)

Task	Description	Status
MCP server	Start MCP server for Claude Desktop	
Error handling	Improve error messages and recovery	
Logging	Structured logging throughout	
Retry logic	Exponential backoff on failures	
CI/CD	GitHub Actions for test + publish	
PyPI publish	Publish to PyPI	
Full test run	Manual testing checklist	

13. Success Metrics

Metric	Target
Installation success rate	> 95%
Sync success rate	> 90%
Time to first sync	< 10 minutes
Data extraction accuracy	> 99%
Per-user attribution accuracy	> 85%
Test coverage	> 80%

Metric	Target
PyPI downloads (first month)	> 100

14. Risks & Mitigations

Risk	Impact	Mitigation
API changes from vendors	Extraction breaks	Version pinning, monitoring
Rate limiting	Sync fails	Exponential backoff, caching
Invalid API keys	Setup fails	Clear error messages, test command
Data duplication	Inaccurate metrics	Dedup by event_id, idempotent uploads
Network issues	Sync interrupted	Local cache, resume on reconnect
Tenant data leak	Security breach	Tenant isolation, no cross-tenant data

15. Future Enhancements (Post-v1)

Feature	Description	Priority
Langfuse extractor	Extract from Langfuse OTel data	High
GitLab extractor	Support GitLab outcomes	Medium
Linear/Jira extractor	Link outcomes to initiatives	Medium
Webhook support	Real-time outcome notifications	Medium

Feature	Description	Priority
Auto-detection	Auto-detect installed AI tools	Low
GUI wizard	Setup wizard with prompts	Low
Docker image	Run MCP in Docker container	Low

Document Version: 1.0.0 | Last Updated: June 2026